

Scalable Multi-Agent Agricultural Advisory Platform: Architecture Design Document

1. SYSTEM ARCHITECTURE (SCALABLE FIRST PRINCIPLES)

The architectural foundation of the agricultural advisory platform is designed to address the unique constraints of rural India: intermittent network connectivity, low digital literacy, and the necessity for highly localized, multimodal data integration.¹ To achieve a highly scalable, production-grade system where the addition of new cognitive agents does not disrupt core orchestration logic, the system employs a Modular Monolith architecture combined with a Dynamic Graph Construction pattern.⁴

The system operates across four highly decoupled layers, ensuring that failures in connectivity or external tool dependencies do not cascade into systemic crashes.

I. Input & Edge Layer (Mobile/Offline)

Operating directly on the client's mobile device, this layer handles multimodal input—specifically voice audio and GPS telemetry. Because continuous internet access cannot be guaranteed in agricultural zones², this layer implements a strict "Offline-First Outbox Pattern".⁵ All voice captures and analytical requests are initially written to a local SQLite database. A background synchronization engine polls the network state; upon detecting connectivity, it transmits the pending queue to the backend using idempotency keys to prevent duplicate processing.⁷ This ensures zero data loss and persistent application usability even in disconnected environments.⁸

II. API Gateway & Orchestration Layer

The backend entry point is an asynchronous application layer managing REST endpoints for batch synchronization and WebSocket connections for real-time streaming.⁹ The core component of this layer is the Dynamic Orchestrator, functioning as a semantic router. Traditional architectures utilize rigid if-else routing statements that must be refactored every time a new feature is added. In contrast, this orchestration layer dynamically reads from an agent registry at startup, constructing the decision graph dynamically.¹⁰ The orchestrator assesses the transcribed user intent, delegates the operational context to the specialized agent, and manages the aggregation of the final response.

III. Agent Layer (Dynamic Registry)

This layer contains the autonomous reasoning engines. Initially populated by the Scheme Navigator Agent and the Crop Sowing Agent, the layer is strictly standardized. Every agent is an independent, encapsulated module adhering to a uniform interface. The architecture supports "Plug-and-Play" extensibility; introducing a new agent merely requires placing its logic module within the designated registry directory and defining its capability schema.¹¹ The dynamic orchestrator automatically incorporates the new agent into its semantic routing table without requiring structural code modifications.⁴

IV. Data & Tool Integration Layer

The foundational layer encompasses local databases and external Application Programming Interfaces (APIs) accessed by the agents to ground their reasoning in empirical data.

- **Vector Datastore:** A PostgreSQL instance utilizing the pgvector extension, storing semantic embeddings of Indian government agricultural schemes and eligibility criteria scraped from the myScheme portal.¹²
- **Geospatial & Climatic Services:** Integrations with the Indian Space Research Organisation (ISRO) Bhuvan Web Map Services (WMS) for soil mapping ¹³ and the Indian Meteorological Department (IMD) / OpenWeather APIs for real-time and historical climatic data.¹⁴
- **Speech Processing Services:** Digital India BHASHINI APIs providing localized Speech-to-Text (STT) and Text-to-Speech (TTS) capabilities tailored to regional Indian dialects.¹⁵

2. AGENT ORCHESTRATION DESIGN (CRITICAL)

The selection of the multi-agent orchestration framework dictates the scalability, determinism, and maintainability of the entire platform.

Framework Selection: LangGraph

The architecture mandates the implementation of **LangGraph**, decisively excluding CrewAI and AutoGen for this production-grade deployment.

Evaluation Criteria	LangGraph	CrewAI	AutoGen
Architectural Paradigm	Stateful, Directed Acyclic Graph (DAG) with cyclic capabilities. ¹⁷	Role-based, sequential team execution. ¹⁸	Conversational, emergent multi-agent dialogue. ¹⁹

Control & Determinism	High. Explicit state management and exact control over conditional routing. ²⁰	Moderate. Excellent for linear tasks, but struggles with complex, branched logic. ²¹	Low. Conversational loops can lead to unpredictable execution paths and high token usage. ²²
Scalability	High. Scales effectively for complex enterprise systems via isolated state transitions. ¹⁸	Moderate. Limited by parallel execution constraints in complex dependencies. ²¹	Low to Moderate. Token overhead scales exponentially with conversation history. ²²
Production Feasibility	High. Native human-in-the-loop support and "time-travel" state debugging. ²³	Moderate. Highly abstracted, making granular error recovery difficult. ¹⁷	Moderate. Better suited for exploratory research than rigid application logic. ¹⁹

Justification for LangGraph: While CrewAI provides rapid prototyping capabilities suitable for early hackathon stages²⁴, its sequential nature becomes a bottleneck when dealing with highly conditional workflows (e.g., an agent failing to fetch GIS data and needing to execute a localized fallback protocol). AutoGen's conversational model introduces significant latency and token costs, which are detrimental to a platform aiming for cost-efficiency in rural deployments.²² LangGraph treats the workflow as a state machine. It provides granular control over memory, fault tolerance, and routing, making it the only viable choice for a robust, scalable backend.²⁵

Orchestration Design Mechanics

1. Dynamic Agent Registry Pattern To achieve the requirement of adding new agents without modifying core logic, the system utilizes a Dynamic Registry Pattern. Agents are not hardcoded into the graph definition. Instead, a factory function parses an /agents directory during application startup. Each agent module exports an AgentSpec containing its identifier, description, and execution node.¹⁰ The factory function iterates through this registry, dynamically executing graph.add_node() for each discovered agent, effectively building the system topology at runtime.⁴

2. Dynamic Semantic Routing Routing is not keyword-based; it relies on semantic intent classification. The system utilizes LangGraph's Command() API, which allows nodes to explicitly

dictate the execution flow and update the state simultaneously.²⁷ The central Supervisor/Router node receives the user's transcribed query and evaluates it against the aggregated descriptions of all registered agents. The Router outputs a structured decision, yielding a `Command(goto="target_agent_id", update={"current_intent": "..."})`.²⁷ Because the router dynamically pulls agent descriptions from the registry, adding a new agent automatically updates the router's cognitive map without manual prompt engineering.

3. Agent Chaining (Multi-Agent Collaboration) When tasks span multiple domains, agents collaborate by mutating a shared `GraphState`. For example, if a user requests a crop recommendation and a subsidy application, the Router dictates execution to the Crop Sowing Agent. Once the Crop Agent completes its spatial analysis, instead of terminating, it observes the unresolved subsidy intent in the `GraphState`. It then returns a `Command(goto="scheme_navigator_agent")`, passing its crop recommendation as enriched context to the Scheme Agent.¹¹

4. Failure Handling and Fallback Rural deployments are prone to third-party API timeouts (e.g., Bhuvan WMS unavailability). LangGraph manages this by encoding failures directly into the graph. Every agent node is wrapped in a try-except execution block. If an external tool fails after configured retries, the agent catches the exception and returns a `Command(goto="fallback_node")`.²⁶ The fallback node analyzes the error state and synthesizes a graceful, localized audio response (e.g., "The weather service is currently unreachable, but based on historical data...") rather than crashing the application.²⁶

3. AGENT DESIGN STANDARD (PLUG-AND-PLAY)

To guarantee that a new agent can be integrated into the system in under 30 minutes, a strict, standardized contract is enforced across all cognitive units.

Base Agent Interface

All agents must implement a standardized `BaseAgent` Python class, ensuring uniformity in how they interact with the LangGraph state machine.

- **Inputs:** Every agent receives the global `GraphState`. This is a `TypedDict` containing:
 - `messages`: The ongoing sequence of human and AI interactions (managed by `BaseMessage` classes).
 - `telemetry`: A dictionary containing offline-synced context, including latitude, longitude, timestamp, and `device_language`.
 - `extracted_entities`: A persistent dictionary of variables extracted over time (e.g., `farm_size`, `aadhaar_status`).
- **Outputs:** The agent must return a LangGraph `Command` object.²⁸ This object encapsulates the update dictionary (which merges new findings into the `GraphState`) and the `goto` string, routing control back to the Supervisor or directly to the user output

node.²⁷

Tool Abstraction Layer

Agents are decoupled from direct API execution. The architecture utilizes a Tool Abstraction Layer where external integrations are defined as isolated, pure functions decorated with `@tool` (or equivalent schema definitions). An agent's `AgentSpec` simply lists the tools it requires (e.g., `tools=[fetch_soil_data, query_scheme_vector_db]`). The framework handles the parsing of tool schemas, argument injection, and error catching automatically.²⁹

Memory Usage Strategy

- **Short-Term Memory:** Managed natively by LangGraph's state persistence. The messages array maintains the context of the immediate session. When the final response is generated, this short-term context is summarized and cleared to prevent context-window overflow on subsequent requests.¹⁷
- **Long-Term Memory:** Critical operational data (e.g., a farmer's location, previously recommended crops, application statuses) is persisted to a PostgreSQL relational database. Upon application launch, a `state_hydration` node queries the database and injects these historical facts into the initial `GraphState`, allowing the agent to provide highly personalized continuity.³¹

Adding a New Agent in <30 Minutes

1. **Create Module:** Create a new Python file in the `/agents` directory (e.g., `market_price_agent.py`).
2. **Define Tools:** Write standard Python functions for the required APIs (e.g., fetching local mandi prices) and decorate them with `@tool`.
3. **Define Agent Logic:** Instantiate the `create_react_agent` (or a custom graph) utilizing the LLM and the defined tools.³²
4. **Export Spec:** Define and export an `AgentSpec` containing the id, name, description, and the callable node function.¹⁰
5. **Deploy:** The central orchestration factory automatically discovers the new file, registers the node, maps the edges, and updates the semantic router's context.⁴ No centralized routing scripts require modification.

4. TECH STACK (FREE-TIER + MVP OPTIMIZED)

The technology stack is aggressively optimized for a 48-hour hackathon build, prioritizing open-source tooling, generous free tiers, rapid iteration speed, and a direct evolutionary path to enterprise scale.

Component	Selected Technology	Justification (Cost, Speed, Scalability)
Frontend	React Native (Expo)	Cost: Open-source. Speed: Expo allows near-instant testing on physical devices via the Expo Go app, eliminating complex Android Studio setups during a hackathon.³³ Scalability: Compiles to highly performant native binaries. Natively supports SQLite (expo-sqlite) for offline outbox implementations⁵ and background background-sync tasks.
Backend	Python (FastAPI)	Cost: Open-source. Speed: Features automatic OpenAPI documentation and pydantic validation, drastically reducing debugging time. Native support for async/await enables high-throughput concurrent LLM calls.³⁴ Scalability: Easily dockerized and highly performant under load compared to traditional synchronous frameworks.
Database	Supabase (PostgreSQL + pgvector)	Cost: Generous free tier supporting 500MB database space and 2GB bandwidth. Speed: Provides out-of-the-box

		REST APIs if needed, but primarily serves as a robust relational database. Scalability: The native pgvector extension allows both relational user data and vector embeddings to exist in the same cluster, avoiding the complexity and cost of managing a disparate vector database (like Pinecone).³⁵
Hosting	Render (Web Services)	Cost: Offers a free tier for web services. Speed: Connects directly to GitHub for continuous integration and automated deployments. Scalability: Transitions instantly from free-tier to scalable, load-balanced instances via dashboard configurations when moving to production.

5. FRONTEND USER FLOWS (VOICE-FIRST UX)

The user interface explicitly targets agricultural workers with marginal digital literacy. Following established "Text-Free UI" design principles³, the application minimizes textual menus, hierarchical navigation, and complex forms. The primary interaction modality is auditory, supplemented by simple, universally recognized iconography.

Voice Query to Response Flow

1. **Initiation:** The application opens to a single, prominent action button. The user taps the button and speaks their query in their native dialect (e.g., "My crops are turning yellow, what should I do?").
2. **Capture:** The expo-audio module captures the audio payload while the system simultaneously retrieves the active GPS coordinates via expo-location.³³
3. **Transmission:** The audio file and JSON telemetry (GPS, timestamp) are packaged. If the

device is online, the payload is transmitted to the FastAPI backend.

4. **Processing & Playback:** The backend processes the audio, orchestrates the agent logic, and returns a synthesized TTS audio stream. The mobile device automatically plays the response to the user.

Scheme Application Flow

1. **Intent Recognition:** The user verbally requests assistance applying for a subsidy (e.g., PM-Kisan).³⁶
2. **Iterative Data Collection:** The Scheme Navigator Agent assesses the application requirements against the user's stored profile. It streams a voice prompt back: "To apply for PM-Kisan, I need your Aadhaar number. Please read it to me."
3. **Entity Extraction:** The user speaks the number. The backend STT transcribes it, and the LLM extracts the structured numeric entity, updating the GraphState.
4. **Auto-filling:** Once all mandatory fields are collected, the agent triggers the PyMuPDF tool. The system overlays the extracted JSON payload onto the specific coordinates of the government PDF template.³⁷
5. **Confirmation:** The system saves the generated document and verbally confirms successful completion to the farmer.

Farm Location Capture (GPS)

To minimize friction, geospatial context is captured passively. During the initial application onboarding, the user grants location permissions. Subsequently, every voice query payload is enriched with the exact Latitude and Longitude. The Crop Sowing Agent relies on this silent telemetry to automatically query geospatial APIs, delivering hyper-localized advice without forcing the user to manually select their district or block.³⁹

Offline Fallback (Low Connectivity) Strategy

Farms represent harsh environments for connectivity assumptions. The application implements an "Offline-First" architecture using the SQLite outbox pattern.²

1. **Local Queuing:** When a user initiates a voice query without a network connection, the application does not display an error. Instead, the audio file is saved locally, and a record is inserted into the SQLite outbox table containing the file URI, idempotency key, and status (pending).⁵
2. **Graceful Degradation:** A locally cached audio prompt plays: "Your question has been saved. I will find the answer as soon as you have a connection."⁸
3. **Background Synchronization:** A background task continually monitors the network state. Upon detecting connectivity, it sequentially pushes the outbox payloads to the backend. The server processes the requests and returns the TTS responses.
4. **Asynchronous Delivery:** The application receives the responses, caches the incoming

audio, and triggers a local push notification to alert the farmer that their advisory is ready for playback.

6. BACKEND SYSTEM DESIGN

API Structure

The FastAPI backend exposes a streamlined, versioned REST API.

- **POST /api/v1/voice/sync:** The primary endpoint. Accepts a multipart payload containing the audio file and a JSON blob of telemetry (GPS, user ID, idempotency key). Designed to handle both real-time queries and delayed outbox syncs.
- **GET /api/v1/schemes/documents/{user_id}:** Retrieves the URLs for auto-filled PDF application forms generated by the Scheme Agent.
- **GET /health:** Standard health check for Render load balancers.

Monolith vs. Microservices

The system utilizes a **Modular Monolith** design.

- **Justification:** While microservices offer independent scaling, they introduce severe operational complexity, network latency, and deployment overhead completely unsuited for a 48-hour MVP build. A modular monolith maintains strict logical separation internally (e.g., agents are separate Python modules, tools are isolated) but deploys as a single unified process.⁴¹ This ensures simplicity while leaving the architecture highly cohesive and ready to be fractured into microservices at a later enterprise scale.

Caching Strategy

To reduce latency, minimize LLM token consumption, and mitigate the impact of external API rate limits, a multi-tiered caching strategy is implemented.

- **API Response Caching:** Static or slowly changing data (e.g., Bhuvan soil classifications for a specific bounding box, weekly weather averages) are cached using in-memory LRU (Least Recently Used) caches or an external Redis instance if available.
- **Semantic Caching (Future Scope):** Common queries ("What is the PM-Kisan eligibility?") can be intercepted by a semantic cache, returning pre-computed TTS responses without invoking the LLM or STT pipelines.²³

Async Processing

The FastAPI backend leverages asynchronous processing natively. The orchestration layer utilizes `asyncio.gather()` to execute non-dependent agent tools in parallel (e.g., querying the weather API and the soil API simultaneously). For offline outbox synchronization, FastAPI's

BackgroundTasks allow the server to instantly return a 202 Accepted response to the mobile client, subsequently processing the audio through the computationally heavy LangGraph pipeline without holding the HTTP connection open.⁴²

7. DATA SOURCES + RAG IMPLEMENTATION

The credibility of the advisory platform relies entirely on the quality of its underlying data. The system eschews generic LLM knowledge in favor of strict Retrieval-Augmented Generation (RAG) and API integration.⁴³

Real Data Sources

Domain	Source	Data Modality & Usage
Government Schemes	myScheme.gov.in & India.gov.in APIs	Unstructured web data and PDF guidelines detailing eligibility, benefits, and application requirements for PM-Kisan, PMKMY, and state-level subsidies. ⁴⁴
Weather	IMD / OpenWeatherMap APIs	Structured JSON data providing current climatic conditions and historical rainfall averages based on GPS coordinates. ¹⁴
GIS & Soil Data	ISRO Bhuvan Portal (WMS)	Web Map Services (Iulc:BR_LULC50K_1112) providing geospatial vector data on soil texture, depth, and land degradation. ¹³
Crop Suitability	ICAR Datasets / Kaggle Repositories	Structured agricultural datasets defining optimal Nitrogen, Phosphorus, Potassium (NPK), pH, and rainfall thresholds for specific crop yields. ⁴⁷

RAG Implementation Pipeline

1. Ingestion Pipeline Data regarding government schemes is highly unstructured. Scripts continuously parse the myScheme sitemaps and PDF guidelines.¹² This text is processed using a semantic chunker (e.g., LangChain's RecursiveCharacterTextSplitter) optimized to keep contextual blocks (like eligibility criteria lists) intact.

2. Vector Database Usage

The text chunks are passed through an embedding model (e.g., OpenAI text-embedding-3-small) to generate high-dimensional vectors. These vectors are stored in the Supabase PostgreSQL database using the pgvector extension. Crucially, each vector is appended with rich metadata (e.g., {"state": "Maharashtra", "category": "subsidy"}) to allow for precise pre-filtering during retrieval.

3. Retrieval Flow When a user asks about subsidies, the Scheme Navigator Agent invokes its @tool for scheme retrieval. The tool converts the user's localized query into an embedding. It performs a cosine similarity search against the pgvector database, passing the user's GPS-derived state as a metadata filter to prevent retrieving irrelevant regional schemes. The top-K relevant chunks are retrieved and injected into the agent's prompt, constraining the LLM to generate responses strictly based on the authenticated government data.⁴³

8. STT + TTS (FREE / LOW COST)

Achieving conversational fluidity for low-literacy users requires specialized speech models trained on regional dialects, rather than generalized Western models.³

Choice: Bhashini API (Digital India BHASHINI Division)

The system selects the **Bhashini API** for both Speech-to-Text (STT) and Text-to-Speech (TTS) processing.⁹

Justification & Technical Explanation

- **Latency:** Bhashini offers a Streaming WebSocket API specifically optimized for conversational interactions. Unlike standard REST APIs that require the entire audio file to upload before processing begins, the WebSocket approach streams binary audio frames continuously. It features server-side Voice Activity Detection (VAD) and automatic speech segmentation, returning interim transcripts instantly and minimizing the end-to-end conversational latency to near human-like levels.¹⁵
- **Language Support:** Bhashini natively supports 22 official Indian languages and local dialects. It is explicitly trained on Indian accents and vernacular code-switching (e.g., mixing Hindi and English), ensuring significantly higher transcription accuracy for rural demographics compared to generalized models like OpenAI Whisper or AWS

Transcribe.⁵⁰

- **Cost Effectiveness:** Bhashini operates on a highly subsidized pricing model to promote digital inclusion. For individual or non-commercial startup usage, the cost is capped at ₹250 per month for up to 50,000 characters per day.⁵² This acts as a robust, essentially free tier for the MVP phase, vastly undercutting the per-minute billing of commercial cloud providers.
- **Offline Capability Adaptation:** While Bhashini requires an internet connection to process audio, the architecture mitigates this through the mobile application's SQLite outbox. When offline, audio is durably queued locally; upon reconnection, the payloads are batch-processed through Bhashini, maintaining the integrity of the voice-first experience despite infrastructure gaps.⁵

9. LLM STRATEGY (DEMO + SCALE)

Choice: API-Based (OpenAI gpt-4o-mini / Anthropic claude-3-haiku)

For the MVP and hackathon phase, the architecture exclusively utilizes high-speed, API-based Large Language Models (LLMs).

Justification

- **Why API-Based:** Deploying and fine-tuning open-source models (like Llama-3 8B) requires provisioning high-memory GPU instances. This introduces massive infrastructure complexity, latency variability, and prohibitive upfront costs that are antithetical to a 48-hour build.⁵³ Fast, frontier API models offer sub-second inference out of the box.
- **Structured Outputs:** A critical function of the Scheme Navigator Agent is the auto-filling of complex government PDF forms. This requires the LLM to extract conversational data and map it perfectly to a rigid JSON schema. API models like OpenAI offer native "Structured Outputs" guarantees, ensuring the generated JSON perfectly aligns with the PyMuPDF population scripts without requiring brittle regex parsing or error-prone validation loops.³⁸

Cost Tradeoffs

API models operate on a Pay-As-You-Go OpEx model, incurring costs per token. While open-source models lack this per-token variable cost, they possess a high CapEx baseline for server hosting. At the scale of a prototype (and initial production), the compute cost of the API is negligible compared to the engineering velocity gained.⁵¹

Evolutionary Switching Strategy

The LangGraph orchestration layer interfaces with LLMs via LangChain's abstraction classes

(e.g., ChatOpenAI). Because the cognitive logic is decoupled from the model provider, switching to an open-source, locally hosted model (via Ollama or HuggingFace Text Generation Inference) in the future requires modifying a single line of configuration code to utilize ChatOllama instead, preserving the entire graph architecture.³⁴

10. GIS IMPLEMENTATION (DETAILED)

The Crop Sowing Agent provides scientific, localized agricultural advice by bridging mobile telemetry with complex spatial datasets.

Step-by-Step Implementation Workflow

1. **GPS Capture:** The React Native frontend utilizes the expo-location library. During a voice query, the device's current Latitude and Longitude are silently appended to the metadata payload.³³
2. **Dataset Querying (Bhuvan & IMD):**
 - The backend orchestrator receives the coordinates. The Crop Sowing Agent executes a tool to fetch weather data from the OpenWeatherMap API (acting as a proxy for IMD data), retrieving average temperature, humidity, and historical rainfall.¹⁴
 - Simultaneously, the agent executes a geospatial query against the ISRO Bhuvan Web Map Service (WMS) endpoint. Using the provided coordinates, it formats a GetFeatureInfo request with a narrow bounding box (BBOX) against thematic layers such as lulc:BR_LULC50K_1112 and Indian Soil Datasets to extract categorical data on soil texture, depth, and pH classification.⁵⁴
3. **Analytical Processing:**
 - The extracted parameters are processed using the Indian Council of Agricultural Research (ICAR) Crop Suitability logic. The system applies the Analytical Hierarchy Process (AHP) matrix, calculating a suitability index based on specific weights: Rainfall (0.31), Temperature (0.22), Soil Depth (0.15), Soil Drainage (0.11), and Soil Texture (0.08).⁵⁶
 - For the MVP, these parameters (N, P, K, pH, Temp, Rainfall) can alternatively be passed through a lightweight Random Forest classification model trained on the Kaggle Crop Recommendation dataset to predict the optimal yield.⁴⁷
4. **Recommendation Synthesis:** The raw analytical output (e.g., "Optimal: Groundnut, Confidence: 0.92")⁵⁸ is returned to the LLM. The agent synthesizes this technical data into a conversational, localized audio response (e.g., "Based on the sandy soil and upcoming rains in your area, groundnuts are your best option this season.") and routes the response for TTS generation.

11. AGENT INTERACTION FLOW (STEP-BY-STEP)

The following illustrates a complete execution trace, demonstrating multi-agent collaboration and offline resilience.

Scenario: A farmer in Maharashtra with no internet connection attempts to record a query.

1. **Capture (Offline):** The user presses the microphone and speaks in Marathi: *"I have two acres in Pune. What crop is best right now, and how do I apply for a subsidy?"* The app detects no network. The audio and GPS coordinates (Lat/Lon) are inserted into the local SQLite outbox table with a pending status.⁵ A local TTS cached audio plays: *"Your request is saved."*
2. **Synchronization:** Hours later, the device detects a cellular connection. The background worker drains the outbox and transmits the multipart payload to the `/api/v1/voice/sync` endpoint.⁸
3. **Transcription:** The backend streams the audio to the Bhashini API, returning the Marathi transcript and an English translation.⁹
4. **Semantic Routing:** The LangGraph Supervisor evaluates the intent. It recognizes a dual requirement (Crop + Scheme). It yields `Command(goto="crop_sowing_agent")`.²⁸
5. **Agent 1 (Crop Sowing):**
 - The agent extracts the GPS from the telemetry state.
 - It invokes the GIS tools, querying Bhuvan WMS and Weather APIs.⁵⁵
 - It calculates the ICAR suitability index⁵⁶, determining Soybean is optimal.
 - It updates the GraphState with `{"recommended_crop": "Soybean"}` and yields `Command(goto="scheme_navigator_agent")` to continue the pipeline.²⁷
6. **Agent 2 (Scheme Navigator):**
 - The agent observes the updated state (Location: Pune, Crop: Soybean).
 - It executes the RAG tool against the pgvector database, filtering for Maharashtra agricultural subsidies.⁴³
 - It extracts PM-Kisan details and synthesizes a combined response.
7. **Response Delivery:** The final text is passed to Bhashini TTS. The synthesized Marathi audio file is returned to the mobile client. The app caches the audio, marks the outbox item as synced, and fires a local push notification alerting the farmer that their advisory is ready to play.

12. DEPLOYMENT PLAN

Local Demo Setup (Hackathon): The development environment is containerized to ensure identical setups across team machines. A `docker-compose.yml` file provisions the PostgreSQL database (with pgvector initialized) and the FastAPI backend. The React Native frontend is executed via the Expo Go application on physical testing devices, connecting to the local backend through an Ngrok tunnel.³³

Free-Tier Cloud Deployment:

For the production MVP presentation, the infrastructure is deployed to zero-cost cloud providers:

- **Database:** Supabase hosts the PostgreSQL instance, managing user data, the vector embeddings, and connection pooling.
- **Backend:** The FastAPI application is deployed as a Web Service on Render's free tier. Render automatically builds and deploys the container from the GitHub repository upon merges to the main branch.
- **Integrations:** API credentials for Bhashini, OpenWeather, and OpenAI are securely injected into Render via Environment Variables.

13. 48-HOUR BUILD PLAN (HACKATHON EXECUTION)

The development schedule is strictly segmented to guarantee a functional, demonstrable product within 48 hours, prioritizing critical paths over peripheral features.

Timeline	Phase	Key Deliverables & Milestones
Hours 1-8	Core Infrastructure & Edge Layer	Initialize React Native Expo. Implement expo-audio and expo-location. Design the SQLite outbox schema (id, payload, status). Scaffold the FastAPI backend and test the bidirectional HTTP/multipart pipeline. ⁵
Hours 9-16	STT/TTS & Orchestration Backbone	Integrate Bhashini API WebSockets for real-time transcription and synthesis. Build the baseline LangGraph architecture, establishing the GraphState and the Supervisor router using the Command() API. ⁹
Hours 17-24	Scheme Navigator & RAG	Provision Supabase. Scrape and chunk 5-10 PM-

	Data	Kisan/myScheme guidelines. Generate embeddings and store in pgvector. ¹² Build the Scheme Agent and verify successful vector retrieval and prompt injection.
Hours 25-32	Crop Sowing Agent & GIS Integration	Construct the Crop Agent. Integrate OpenWeatherMap API. Formulate the Bhuvan WMS GetFeatureInfo requests for soil data. ⁵⁵ Implement a simplified Python function representing the ICAR AHP matrix. ⁵⁶
Hours 33-40	PDF Automation & Multi-Agent Chaining	Integrate PyMuPDF. Configure the Scheme Agent to utilize OpenAI Structured Outputs to generate exact JSON mappings, successfully auto-filling a sample government PDF. ³⁷ Test the full routing chain (Crop -> Scheme -> Output).
Hours 41-48	Deployment, UX Polish, & Demo Prep	Deploy backend to Render. Ensure the React Native UI adheres to Text-Free, voice-first principles (large iconography, single-button interaction). Conduct field testing simulating network drops to verify the outbox sync recovery protocol. ⁸

Works cited

1. User Interface Design for Low-literate and Novice Users: Past, Present and Future, accessed on April 25, 2026, <https://www.emerald.com/fthci/article/8/1/1/1320900/User-Interface-Design-for-Low-literate-and-Novice>
2. Offline-First Patterns for Rural Connectivity: Buffering, Sync, and Freshness SLAs - LoteAI - Precision Agriculture for Midwest Farms, accessed on April 25, 2026, <https://loteai.app/resources/offline-first-farm-connectivity>
3. UIs for Low-Literate Users - Microsoft Research, accessed on April 25, 2026, <https://www.microsoft.com/en-us/research/project/uis-low-literate-users/>
4. Dynamic Graph Creation at Runtime - LangGraph - LangChain Forum, accessed on April 25, 2026, <https://forum.langchain.com/t/dynamic-graph-creation-at-runtime/1387>
5. React Native offline sync with SQLite queue - DEV Community, accessed on April 25, 2026, https://dev.to/sathish_daggula/react-native-offline-sync-with-sqlite-queue-4975
6. How to Implement Offline-First Architecture in React Native - OneUptime, accessed on April 25, 2026, <https://oneuptime.com/blog/post/2026-01-15-react-native-offline-architecture/view>
7. React Native offline-first: conflict-safe SQLite sync - DEV Community, accessed on April 25, 2026, https://dev.to/sathish_daggula/react-native-offline-first-conflict-safe-sqlite-sync-549a
8. Offline-First Architecture:. Building Apps That Work When Nothing... | by Aditya Athavale | Stackademic, accessed on April 25, 2026, <https://blog.stackademic.com/offline-first-architecture-5cb12314af75>
9. Bhashini.ai API Docs, accessed on April 25, 2026, <https://www.bhashini.ai/docs>
10. Dynamically create graph from config : r/LangChain - Reddit, accessed on April 25, 2026, https://www.reddit.com/r/LangChain/comments/1i10x2g/dynamically_create_graph_from_config/
11. Mastering-Agentic-Design-Patterns-with-LangGraph/README.md at main - GitHub, accessed on April 25, 2026, <https://github.com/MahendraMedapati27/Mastering-Agentic-Design-Patterns-with-LangGraph/blob/main/README.md>
12. shrijoyan/gov_myscheme · Datasets at Hugging Face, accessed on April 25, 2026, https://huggingface.co/datasets/shrijoyan/gov_myscheme
13. How to use WMS services - Bhuvan Wiki - NRSC, accessed on April 25, 2026, https://bhuvan.nrsc.gov.in/wiki/index.php/How_to_use_WMS_services
14. Bhuvan - IMD Weather Products, accessed on April 25, 2026, <https://bhuvan-app1.nrsc.gov.in/imd/>
15. Bhashini.ai Streaming Speech-to-Text (STT) WebSocket, accessed on April 25, 2026, <https://www.bhashini.ai/docs/stt>
16. Bhashini, accessed on April 25, 2026, <https://bhashini.gov.in/>
17. Technical Comparison of AutoGen, CrewAI, LangGraph, and OpenAI Swarm | by

Omar Santos | Artificial Intelligence in Plain English, accessed on April 25, 2026, <https://ai.plainenglish.io/technical-comparison-of-autogen-crewai-langgraph-and-openai-swarm-1e4e9571d725>

18. CrewAI vs LangGraph vs AutoGen: Choosing the Right Multi-Agent AI Framework, accessed on April 25, 2026, <https://www.datacamp.com/tutorial/crewai-vs-langgraph-vs-autogen>
19. Battle of AI Agent Frameworks: CrewAI vs LangGraph vs AutoGen | by Vikas Kumar Singh, accessed on April 25, 2026, https://medium.com/@vikaskumarsingh_60821/battle-of-ai-agent-frameworks-langgraph-vs-autogen-vs-crewai-3c7bf5c18979
20. LangGraph — Architecture and Design | by Shuvrajyoti Debroy | Medium, accessed on April 25, 2026, <https://medium.com/@shuv.sdr/langgraph-architecture-and-design-280c365aaf2c>
21. LangGraph vs CrewAI: Let's Learn About the Differences - ZenML Blog, accessed on April 25, 2026, <https://www.zenml.io/blog/langgraph-vs-crewai>
22. CrewAI vs LangGraph vs AutoGen: 2026 Comparison - AgentCenter, accessed on April 25, 2026, <https://agentcenter.cloud/blogs/crewai-vs-langgraph-vs-autogen>
23. Mastering Agents: LangGraph Vs Autogen Vs Crew AI - Galileo AI, accessed on April 25, 2026, <https://galileo.ai/blog/mastering-agents-langgraph-vs-autogen-vs-crew>
24. First hand comparison of LangGraph, CrewAI and AutoGen | by Aaron Yu - Medium, accessed on April 25, 2026, <https://aaronyuqi.medium.com/first-hand-comparison-of-langgraph-crewai-and-autogen-30026e60b563>
25. AI Agent Frameworks Compared: LangGraph vs CrewAI vs AutoGen (2026) - PE Collective, accessed on April 25, 2026, <https://pecollective.com/blog/ai-agent-frameworks-compared/>
26. AutoGen vs. CrewAI vs. LangGraph vs. OpenAI Multi-Agents Framework - Galileo AI, accessed on April 25, 2026, <https://galileo.ai/blog/autogen-vs-crewai-vs-langgraph-vs-openai-agents-framework>
27. A Beginner's Guide to Dynamic Routing in LangGraph with Command() | by Damilola Oyedunmade | AI Engineering BootCamp | Medium, accessed on April 25, 2026, <https://medium.com/ai-engineering-bootcamp/a-beginners-guide-to-dynamic-routing-in-langgraph-with-command-2c8c0f3ef451>
28. Dynamic Routing in LangGraph with Command() - YouTube, accessed on April 25, 2026, <https://www.youtube.com/watch?v=5Wpsnw1oIXE>
29. Are dynamic tool lists allowed when using create_agent? - Talking Shop - LangChain Forum, accessed on April 25, 2026, <https://forum.langchain.com/t/are-dynamic-tool-lists-allowed-when-using-create-agent/1920>
30. A Comprehensive Guide to LangGraph: Managing Agent State with Tools - Medium, accessed on April 25, 2026, <https://medium.com/@o39joey/a-comprehensive-guide-to-langgraph-managing-agent-state-with-tools-ae932206c7d7>

31. Building Agentic Workflows with LangGraph and Granite - IBM, accessed on April 25, 2026, <https://www.ibm.com/think/tutorials/build-agentic-workflows-langgraph-granite>
32. Agents - Docs by LangChain, accessed on April 25, 2026, <https://docs.langchain.com/oss/python/langchain/agents>
33. Audio (expo-audio) - Expo Documentation, accessed on April 25, 2026, <https://docs.expo.dev/versions/latest/sdk/audio/>
34. Langgraph vs CrewAI vs AutoGen vs PydanticAI vs Agno vs OpenAI Swarm : r/LangChain, accessed on April 25, 2026, https://www.reddit.com/r/LangChain/comments/1jpk1vn/langgraph_vs_crewai_vs_autogen_vs_pydanticai_vs/
35. How to Build a LangGraph Research Agent that Embeds Dynamic Charts via MCP Apps (CopilotKit & Tako) - DEV Community, accessed on April 25, 2026, <https://dev.to/copilotkit/how-to-build-a-langgraph-research-agent-that-embeds-dynamic-charts-via-mcp-apps-copilotkit-tako-3ihi>
36. Ministry of Agriculture and Farmers' Welfare, accessed on April 25, 2026, <https://agriwelfare.gov.in/en/FarmersWelDiv>
37. Automating PDF Form Filling with AI: A Python Implementation - Kite Metric, accessed on April 25, 2026, <https://kitemetric.com/blogs/automating-pdf-form-filling-with-ai-a-python-implementation>
38. Using GPT-4-Turbo to fill out complex PDF forms - API - OpenAI Developer Community, accessed on April 25, 2026, <https://community.openai.com/t/using-gpt-4-turbo-to-fill-out-complex-pdf-forms/722020>
39. A fusion approach using GIS, green area detection, weather API and GPT for satellite image based fertile land discovery and crop suitability - PMC, accessed on April 25, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC11247080/>
40. Voice4All: Voice-based social media for rural developing regions - Stanford HCI Group, accessed on April 25, 2026, <https://hci.stanford.edu/research/voice4all/>
41. Information for Developers - Bhuvan Wiki - NRSC, accessed on April 25, 2026, https://bhuvan.nrsc.gov.in/wiki/index.php/Information_for_Developers
42. Build multi-agent systems with LangGraph and Amazon Bedrock | Artificial Intelligence, accessed on April 25, 2026, <https://aws.amazon.com/blogs/machine-learning/build-multi-agent-systems-with-langgraph-and-amazon-bedrock/>
43. Structured Outputs: Making LLMs Reliable for Document Processing | by Nick Hagar, accessed on April 25, 2026, <https://generative-ai-newsroom.com/structured-outputs-making-llms-reliable-for-document-processing-c3b6b2baed36>
44. myScheme National Platform - Vikaspedia - Digital governance, accessed on April 25, 2026, <https://en.vikaspedia.in/viewcontent/e-governance/digital-india/myscheme-national-platform?lgn=en>
45. myScheme, accessed on April 25, 2026, <https://www.myscheme.gov.in/>
46. Indian Geo Platform of ISRO - Bhuvan - NRSC, accessed on April 25, 2026, <https://bhuvan.nrsc.gov.in/home/index.php>

47. Crop Recommendation Dataset - Kaggle, accessed on April 25, 2026, <https://www.kaggle.com/datasets/atharvaingle/crop-recommendation-dataset>
48. "Crop Recommendation System using Soil and Weather Content", accessed on April 25, 2026, <http://www.ir.juit.ac.in:8080/jspui/bitstream/123456789/11463/1/Crop%20Recommendation%20System%20using%20Soil%20and%20Weather%20Content.pdf>
49. Inclusive Voice-AI for Rural India: A Deep Learning Framework for Dialectal and Low- Resource Adaptation - IJFMR, accessed on April 25, 2026, <https://www.ijfmr.com/papers/2026/1/68629.pdf>
50. Bhashini AI Solutions, accessed on April 25, 2026, <https://www.bhashini.ai/>
51. STT & TTS Pricing Comparison 2025 | Speech-to-Text & Text-to-Speech Costs | edesy.in, accessed on April 25, 2026, <https://edesy.in/ai-voice-assistant/compare/stt-tts-pricing>
52. Server Subscription - Bhashini AI Solutions, accessed on April 25, 2026, <https://www.bhashini.ai/speak/subscription>
53. Financial PDF data extraction with specific JSON schema : r/LocalLLM - Reddit, accessed on April 25, 2026, https://www.reddit.com/r/LocalLLM/comments/1mbhell/financial_pdf_data_extraction_with_specific_json/
54. Agriculture, Water, Soil and Vegetation Data Products - NRSC, accessed on April 25, 2026, <https://bhuvan-app1.nrsc.gov.in/2dresources/bhuvanstore2.php>
55. Querying WMS GetFeatureInfo with known latitude and longitude [closed], accessed on April 25, 2026, <https://gis.stackexchange.com/questions/102169/querying-wms-getfeatureinfo-with-known-latitude-and-longitude>
56. Crop suitability analysis for the coastal region of India through fusion of remote sensing, geospatial analysis and multi-criteria decision making - PMC, accessed on April 25, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC11906854/>
57. Crop Recommendation Using Weather and Soil Content - IJSREM, accessed on April 25, 2026, <https://ijsrem.com/download/crop-recommendation-using-weather-and-soil-content/>
58. Re-evaluating soil moisture-based drought criteria for rainfed crops in peninsular India, accessed on April 25, 2026, <https://www.frontiersin.org/journals/environmental-science/articles/10.3389/fenvs.2024.1319912/full>
59. Designing voice user interface design: how to guide users through words alone - Tangivel, accessed on April 25, 2026, <https://www.tangivel.com/articles/designing-voice-user-interface-designs-how-to-guide-users-through-words-alone>